

Criterio de modularizacion de servicios y tests

Regla general para mantener los modulos del backend faciles de navegar, revisar y ampliar sin crear archivos innecesarios.

Idea central

El codigo se divide por capacidad de negocio. Los tests se dividen por comportamiento observable.

1. Servicios backend

Cada modulo empieza con un unico service.py. No se crean varios servicios desde el primer dia ni se divide un archivo solamente porque parece largo.

Se divide cuando se cumplen las dos condiciones:

- 1 Existen al menos dos capacidades de negocio independientes.
- 2 El archivo supera aproximadamente 500 lineas o 15 operaciones publicas.

Ejemplo aplicado a Inscripciones

Antes, altas, movimientos, opciones y admisiones convivian en un unico archivo de mas de 850 lineas. Ahora se organizan por capacidades reconocibles para el negocio:

```
inscripciones/  
matriculas_service.py # altas, reinscripciones, cambios y bajas  
opciones_service.py  # opciones para formularios  
admisiones_service.py # solicitudes, etapas y documentacion
```

Como se eligen los limites

Buena division	Division incorrecta
Admisiones, Matriculas, Movimientos	Lecturas, Escrituras, Helpers
Vocabulario y reglas propias	Separacion puramente tecnica

2. Tests backend

Los tests se separan por el comportamiento que verifican. No tienen que copiar exactamente la estructura interna de los servicios.

```
tests/inscripciones/  
test_listado.py  
test_altas.py  
test_movimientos.py  
test_opciones.py  
test_admisiones.py  
factories.py  
conftest.py
```

Responsabilidad de cada archivo

Archivo	Contenido
test_*.py	Casos agrupados por comportamiento observable.
factories.py	Builders y datos de prueba reutilizados por varios archivos.
conftest.py	Fixtures de Pytest, como cliente y sesion de base de datos.

3. Reglas practicas

- Un modulo pequeno y cohesivo conserva un solo service.py y un solo archivo de tests.
- No se modulariza por anticipacion ni por una cantidad arbitraria de archivos.
- El router importa cada servicio directamente; no queda un service.py que solo reexporte funciones.
- Un helper se extrae solamente si lo usan dos o mas servicios; si pertenece a un solo flujo, permanece privado.
- La refactorizacion no cambia endpoints ni reglas de negocio: reorganiza el codigo y los mismos tests deben seguir pasando.

Codigo: capacidad de negocio | Tests: comportamiento observable